

1. What is DbContext and what does it do in your project?

DbContext connects my application to the database and allows me to query and save data

2. What is DbSet and how does it map to database tables?

DbSet represents a collection of entities that maps to a database table. For example, DbSet<Student> Students maps the Student entity to the Students table

3. What is a migration and why do we use it?

A migration tracks database schema changes and applies them to the database

4. What is the difference between entity and DTO?

An Entity represents the data model used by the application and database. A DTO, or Data Transfer Object, is used to transfer only the required data between the API and the client. In my project, Enrollment is an entity, while CreateEnrollmentRequest and EnrollmentDetailsResponse are DTOs

5. Why should APIs not return EF entities directly?

To avoid exposing database details and unnecessary data

6. What is a foreign key?

A foreign key connects related tables. Enrollment has StudentId and TrainingTrackId

7. Explain the relationship between Student, TrainingTrack and Enrollment.

A Student can have many Enrollments, and a TrainingTrack can have many Enrollments. Enrollment connects the Student and the TrainingTrack

8. Why is Enrollment a join entity instead of a simple many-to-many?

Enrollment is a join entity because the relationship between Student and TrainingTrack contains additional data. For example, Enrollment stores EnrollmentDate, Status, ProgressPercentage, and FinalResult. So it is not just a simple relationship between two entities

9. What is Include and when did you use it?

Include is used in Entity Framework Core to load related entities along with the main entity. I used it in the original enrollment code to load related Student, TrainingTrack,

and Payments data. In the refactored code, I mainly use projection with Select when I only need specific fields

10. What is Select projection and why is it useful?

It selects only the required fields, which reduces unnecessary data

11. What is pagination and why does an API need it?

Pagination means returning data in smaller pages instead of returning all records at once. In my project, I use pageNumber, pageSize, Skip, and Take for enrollments. This helps reduce the amount of data returned and improves API performance when the database contains many records

12. How did you prevent duplicate active enrollments?

Before creating an enrollment, I check if the same student is already enrolled in the same training track. I use AnyAsync with the StudentId and TrainingTrackId, while ignoring deleted and cancelled enrollments. If an active enrollment already exists, I return a validation error

13. How did you protect track capacity?

Before creating an enrollment, I count the active enrollments for the selected training track using CountAsync. Then I compare the count with the track's Capacity. If the active enrollments have reached the capacity, the new enrollment is rejected

14. How did you handle payment validation?

I validate that the payment amount is greater than zero, the payment method is provided, and the reference number is provided. I also check that the enrollment exists and that the reference number is unique. Finally, when updating payment status, I allow only valid statuses such as Pending, Paid, Failed, and Refunded

15. What is soft delete and why did you use it?

Soft delete means marking a record as deleted instead of physically removing it from the database. In my project, I use IsDeleted and DeletedAt for this. This keeps the original data available while normal queries filter out deleted records

16. What is the difference between local database and remote database?

A local database runs on my development machine and is mainly used during development and testing. A remote database runs on a server or cloud environment and can be accessed by the deployed application. The application uses a connection string to connect to the appropriate database

17. How did you configure the production connection string?

I store it in the production environment and read it through configuration

18. Why should connection strings not be pushed to GitHub?

Connection strings may contain sensitive information such as database usernames, passwords, or server details. If they are pushed to GitHub, they could be exposed. They should be stored securely using environment variables, User Secrets, or deployment platform secrets

19. What is the hardest bug you faced in deployment?

The hardest deployment bug in my project was related to connecting the deployed API to the remote database. I had to verify the production connection string, database availability, and configuration to make sure the API could communicate with the remote database correctly

20. If you had one more week, how would you improve the system?

I would add authentication, better logging, and more tests

21.